01 / 구현 기획의 목적

평가 기준에서 구현·검증까지

제공된 Capstone 평가표와 Harness Summary를 설계 요구사항으로 반영한 구현 초안이다.

서비스와 사업 대상

B2B 리서치 업무 지원 서비스다. 증권사·자산운용사 등의 리서치 담당자가 정정 전후 공급계약 공시와 기존 분석 메모를 등록하면, 근거가 연결된 문장별 영향 검토표와 수정 초안을 받는다. 사용자가 승인한 사실·계산 수정만 새 메모 버전에 반영한다.

서비스를 쓰는 구체적 상황

120억 원 계약이 90억 원으로 정정되면, 기존 메모의 계약금액과 매출액 대비 비율을 점검한다. 종료일 변경은 일정 문장을 수정할 수 있지만 매출 인식 전망은 해석 재검토로 남긴다. 사람이 변경 사항을 미리 작성할 필요는 없다.

제품 차별성과 사업 가설

공시 요약을 넘어 '내 문서의 어떤 주장을 왜 바꿔야 하는가'를 추적하고, 승인·버전 이력까지 연결한다. 조직 구독형 서비스를 가정하며 검토 시간과 수정 누락 감소를 고객 가치 가설로 검증한다. 시장성이나 시간 절감 성과는 아직 검증되지 않았다.

설계 원칙

Agent = Model + Harness. 모델은 다음 조회 행동과 수정 초안을 제안하고, Harness가 도구 실행·한도·기록·승인·복구를 통제한다. 이 문서의 신규 기능은 목표 설계이며 현재 구현 완료를 의미하지 않는다.

02 / 평가 항목 매핑

100점 기본 항목 + 10점 가산점

각 항목은 설계 설명뿐 아니라 실제 코드·테스트·시연 증거로 입증한다.

Biz. 아이템 10점 / 아키텍처 설계 10점

B2B 사용 시나리오·고객 효용 가설·입출력 예시를 제시한다. UI/API, Agent, MCP, 저장소, 인증 경계를 도식화한다. ISO/IEC 25010을 고려한 품질 목표와 측정 방법은 10쪽에 정의한다.

AI Agent 설계 15점 / 구현 15점

명시적 StateGraph, 조건부 분기, 제한된 조사 반복, interrupt/resume, 영속 체크포인터를 구현한다. 모델의 구조화 행동 제안 → 검증된 MCP 실행 → 근거 관찰 → 재판단을 trace로 증명한다.

MCP 설계·구현 20점

business-mcp와 rag-mcp를 별도 책임으로 구성한다. 도구 설명·입력 스키마·권한·오류 계약을 정의하고 Agent가 연결·호출한다. 정상 응답뿐 아니라 인증 실패·타임아웃·없는 근거·문서 충돌을 검증한다.

Harness 20점 / 인증 10점

Instructions, Tools, Context, State·Memory, Agent Loop, Guardrails, Recovery, Observability의 구현 위치와 검증을 연결한다. 사용자 로그인, 조직별 데이터 격리, 도구 권한과 서버 측 승인 검증을 적용한다.

대기열 가산점 10점

RabbitMQ 기반 분석 작업 큐, 실패 재시도, DLQ, 작업 중복 방지와 진행 상태 조회를 추가한다. 가산점 기능이며 핵심 분석·승인 경로를 먼저 완성한다.

03 / 목표 아키텍처

서비스·Agent·MCP의 책임 분리

직접 구성한 StateGraph 방식으로 구현한다. 아래 구성은 추가 개발을 포함하는 목표 구조다.



모든 도구 요청에 검증된 사용자·조직 컨텍스트를 전달한다. MCP 서버는 Agent의 요청과 별도로 권한을 재검증한다. 원문 내용은 데이터로 취급하고 실행 지시로 해석하지 않는다.

04 / State 및 데이터 계약

무엇을 저장해야 안전하게 재개할까

원문 전체를 State에 복제하지 않고 고정된 문서 참조와 버전·해시를 저장한다.

ReviewState의 주요 필드

식별: review_id, thread_id, tenant_id, actor_id
입력: original_doc_id, corrected_doc_id, source_kind, document_hashes, memo_id, memo_version
분석: contract_relation, field_changes, claims, evidence_ids, proposals, unresolved_reasons
실행: next_action, action_history, call_count, deadline, question_id, status
승인: proposal_version, evidence_set_hash, approval_id, save_key

구조화 행동과 도구 응답

모델 행동은 LOAD_RELATED_FILING, SEARCH_EVIDENCE, READ_EVIDENCE_BLOCK, ASK_USER, FINISH 중 하나로 제한한다. action, target_id, reason을 구조화 출력한다. Harness는 대상·권한·중복·예산을 검사한 후 실행한다. 도구 응답은 data, evidence_refs, source_kind, fetched_at 또는 code, retryable, message로 통일한다.

업무 저장 모델

filing, evidence_block, memo, memo_version, review, claim, proposal, approval, audit_event를 분리한다. tenant_id를 업무 레코드에 포함한다. proposal은 수정 원문 범위·원문 해시·대체 문장·근거 ID를 보유한다. 중복 범위와 다른 버전의 수정은 저장 전에 차단한다.

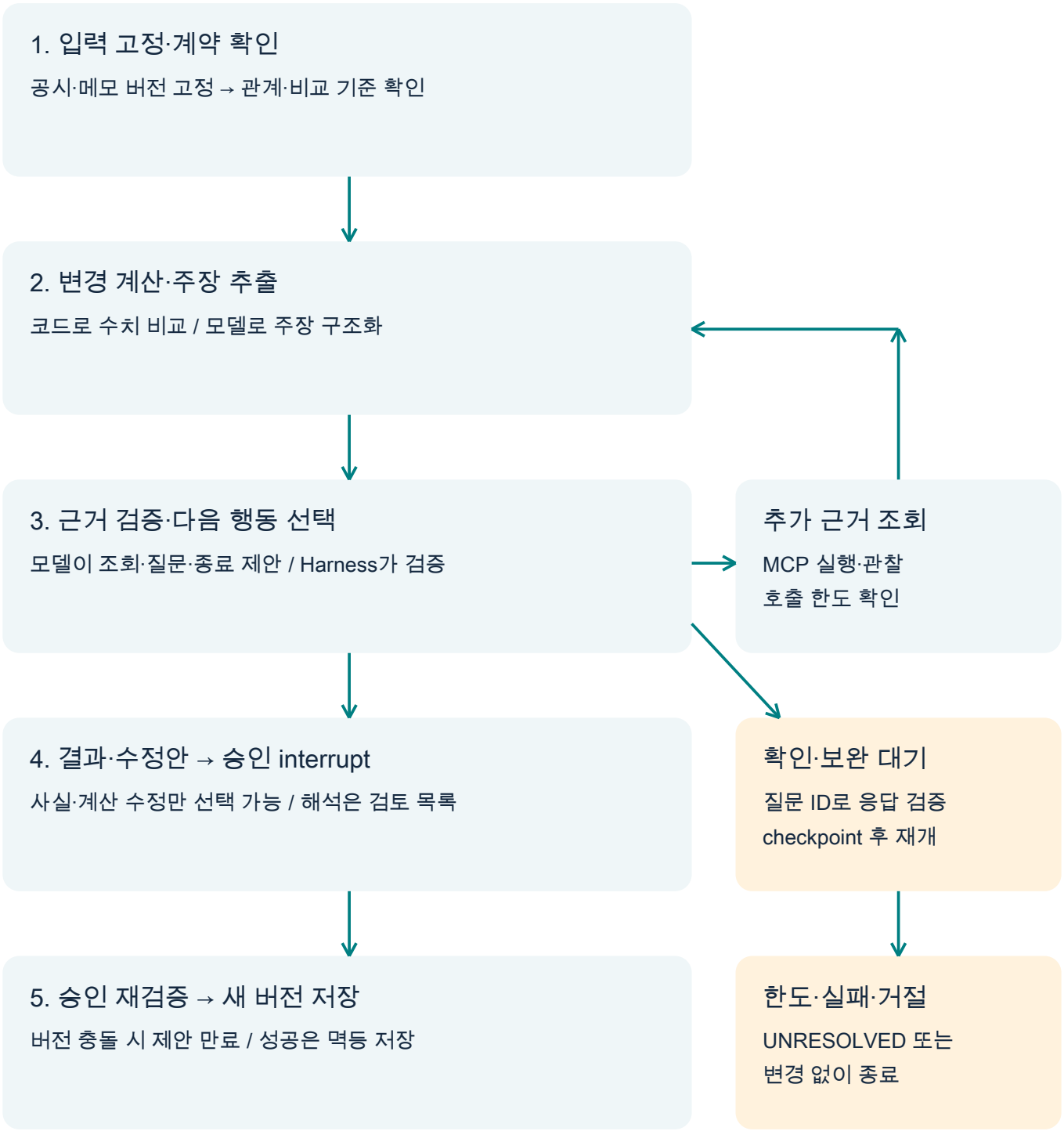
메모리·컨텍스트 정책

단기 실행 상태는 체크포인터, 장기 업무 기록은 DB가 담당한다. 모델에는 현재 주장과 관련 근거만 전달하고 조직·검토 범위를 벗어난 검색을 금지한다. 과거 대화가 공시 원문이나 승인 기록을 대체하지 않도록 한다.

05 / Agent 흐름도

조회·분기·반복·승인·재개

단순 순차 실행을 넘어 근거 부족에 따라 추가 행동을 선택하고 검토 상태를 보존한다.



예산 초안: LLM 8회, 조회 도구 12회, 동일 행동·대상 2회, 실행 120초. 한도는 설정으로 관리한다. 승인 대기 시간은 실행 제한에서 분리한다. 보완 자료가 바뀌면 영향받는 판단과 제안을 무효화한다.

06 / MCP 인터페이스

업무 도구와 근거 검색 도구

프로젝트 전용 MCP 서버 2종을 구현한다. 아래 이름과 스키마는 설계안이다.

business-mcp: 공시·메모 업무

list_filings(company_id, period, cursor): 공시 후보·페이지 정보 반환
get_filing(filing_id): 허용된 공급자에서 확보한 원문 참조·출처·해시 반환
read_memo(memo_id, version): 권한 있는 조직 메모 반환
apply_approved_revision(review_id, approval_id, expected_version, save_key): 승인 검증 후 새 버전 반환. destructiveHint=true로 변경 도구임을 표시한다.

rag-mcp: 검색과 인용

search_evidence(review_id, query, doc_ids, top_k): 해당 검토에서 접근 가능한 근거 후보 반환
read_evidence_block(evidence_id, expected_hash): 실제 원문 블록·위치·문서 해시 반환
검색 결과는 확정 근거가 아니다. 서버 검증기가 필드 일치와 원문 위치를 확인하며, 의미 적합성은 평가 데이터로 별도 검증한다.

도구 선택과 실행 경계

도구별 description에 사용 시점·필수 입력·금지 조건·반환 형식을 명시한다. Agent는 관련 공시나 근거 조회를 선택하되, 쓰기 도구 호출은 승인 전이 노드에서만 허용한다. 모델이 tenant_id나 approved=true를 만들어 권한을 획득할 수 없도록 서버 컨텍스트를 사용한다.

오류·예외 계약

UNAUTHORIZED/FORBIDDEN: 재시도 없이 중단.
NOT_FOUND/UNSUPPORTED_FORMAT: 보완 요청 또는 판단 보류.
RATE_LIMIT/TIMEOUT: 읽기만 지수 백오프·최대 2회 재시도.
VERSION_CONFLICT: 기존 제안 만료 및 재분석. 쓰기 응답 유실은 save_key 조회 후 결과 복구한다.

07 / Harness 8개 요소

모델의 제안을 실행 가능한 업무로 통제

제공된 Harness Summary의 8개 요소를 코드 책임과 확인 시나리오로 연결했다.

Instructions / Tools

버전 관리되는 지시문에 출처 우선·해석 자동 수정 금지·행동 형식을 정의한다. MCP 도구는 등록된 스키마·허용 목록으로만 노출한다. 확인: 원문에 승인 우회 지시를 넣어도 도구 권한이나 저장 정책이 바뀌지 않는다.

Context / State·Memory

현재 검토의 문서·메모·근거만 구성하고 토큰 예산을 둔다. StateGraph 상태와 영속 체크포인터로 중단·재개한다. 확인: 다른 조직 근거 검색 차단, 재시작 후 동일 thread_id 복원, 오래된 질문 응답 거부.

Agent Loop / Guardrails

행동 제안 → 스키마·권한·예산 검증 → 도구 실행 → 관찰을 반복한다. 수치는 Decimal 계산, 근거 ID와 해시는 서버가 확인한다. 확인: 반복 조회·한도 도달 시 종료, 실패를 NO_IMPACT로 바꾸지 않음, 미검증 수치 수정 차단.

Recovery / Observability

재시도 가능한 읽기 오류만 제한 재시도하고 쓰기는 멱등 키로 복구한다. 노드·도구별 trace와 모델·지시문 버전, 지연·호출량·상태 전이를 기록한다. 확인: 타임아웃, 저장 성공 후 응답 유실, Worker 재실행에서 중복 버전이 없다.

실행 증거 제출

각 요소마다 최소 1개 검증 사례와 trace를 제출한다. 원문·API 키·개인정보를 trace에 무제한 기록하지 않는다. 내부 사고 과정 대신 입력 참조, 선택 행동, 검증 결과, 도구 결과와 상태 전이를 보여준다.

08 / 인증·승인·저장

사람의 승인은 서버가 검증한다

로그인 여부와 문서 변경 권한을 분리하고, 조직 경계를 API와 MCP 모두에 적용한다.

사용자 인증과 권한

OIDC 제공자 연동을 기본 설계로 한다. API는 서명·만료·발급자·대상을 검증하고 서버 세션에 사용자와 조직을 연결한다. 권한은 viewer(조회), analyst(분석 요청), reviewer(수정 승인), admin(조직 관리)로 구분한다. 로컬 개발 인증 대역은 운영 인증과 명확히 분리한다.

승인 전 중단

변경 도구는 destructiveHint=true로 표시한다. 실제 쓰기 전 checkpoint에 검토·메모·제안·근거 버전을 저장하고 interrupt로 사람의 결정을 기다린다. 힌트는 승인 강제 장치가 아니므로 서버의 권한·승인 검증을 별도로 적용한다.

승인과 쓰기의 연결

승인 API는 현재 사용자 권한과 선택한 proposal_id를 검증해 approval_id를 생성한다. 재개 노드는 승인 대상과 최신 메모·근거 버전을 다시 확인한다. 유효한 approval_id가 있는 요청만 MCP 변경 도구를 실행한다. 해석 재검토·판단 보류는 일괄 자동 수정에서 제외한다.

충돌·중복·복구

원본을 보존하고 메모 v2를 생성한다. 승인과 새 버전 저장은 업무 DB 트랜잭션으로 묶고 save_key에 유일 제약을 둔다. 동일 키·동일 요청은 기존 결과를 반환하며 다른 내용은 충돌이다. 그래프 체크포인트와 업무 DB의 단일 트랜잭션을 가정하지 않고 저장 영수증으로 복구한다.

09 / 대기열·API·구현 순서

분석 작업은 비동기, 승인은 명시적으로

RabbitMQ는 가산점 범위다. 큐 도입 전에도 동일한 작업 ID와 상태 계약을 유지한다.

API와 화면

POST /reviews: 입력 참조 등록 후 review_id·job_id 반환.
GET /reviews/{id}: queued/running/waiting_user/waiting_approval/completed/failed 상태 조회.
POST /reviews/{id}/answers: 질문 ID·검토 버전과 보완 응답 제출.
POST /reviews/{id}/approvals: 선택 수정안 승인·거절.
GET /memos/{id}/versions: 권한 있는 메모 버전 조회.

RabbitMQ 처리 설계 [가산점]

API → 작업 큐 → Worker → StateGraph 실행. DB에 작업을 기록한 뒤 발행 누락을 막기 위해 outbox를 사용한다. Worker는 완료·안전한 상태 저장 후 ack한다. 지연 재시도 큐와 DLQ를 두고 최대 3회 후 실패 처리한다. 메시지에는 원문·인증 토큰 대신 job_id와 참조만 담는다.

중복과 대기 상태

큐는 중복 전달될 수 있으므로 job_id 기반 실행 임대와 DB 상태 검사를 사용한다. 승인 대기에서는 메시지를 계속 점유하지 않고 checkpoint 저장 후 ack한다. 사용자 응답 시 재개 작업을 발행한다. 쓰기 중복은 별도로 save_key가 차단한다.

구현 순서와 권장 모듈

① state/schema·검증기 → ② MCP 서버와 오류 계약 → ③ 행동 선택·조사 반복 → ④ checkpoint·interrupt → ⑤ 로그인·승인·버전 저장 → ⑥ trace·평가 → ⑦ RabbitMQ.
권장 경계: api/, agent/, harness/, mcp_business/, mcp_rag/, persistence/, workers/, evaluation/. 기존 파싱·계산 코드는 재사용한다.

10 / 품질·평가·완료 조건

설계가 실제 동작함을 보여주는 증거

ISO/IEC 25010의 품질 관점을 고려한 프로젝트 측정 계획이다. 표준 인증이나 준수 완료를 주장하지 않는다.

품질 목표와 측정

기능 적합성: 사람이 검수한 계약별 정답으로 영향 주장 precision/recall, 분류 혼동표, 근거 적합성 측정.
신뢰성·보안: 재시작 복구, 쓰기 멱등성, 조직 격리, 승인 우회 차단 검증.
성능 효율성·사용성: 분석 지연·호출 수·비용과 사용자 검토 시간 측정.
유지보수성·호환성: 공급자 어댑터 교체, MCP 스키마 계약, 모듈별 오류 격리 검증.

평가 데이터와 목표

계약 단위로 개발/고정 평가를 분리하고 반복 정정은 같은 분할에 둔다. 실제/합성, fake/실제 LLM, 사람 검수/임시 정답을 각각 보고한다. 기존 기획 목표인 recall 90%, precision 85%는 목표치로만 둔다. 보류율·미검토율·표본 수·분모를 함께 보고한다.

필수 검증과 시연

정상 변경 → 모델의 근거 조회 → MCP 도구 실행 → 근거 연결 결과 → 승인 대기 → 부분 승인 → 새 버전 조회를 시연한다. 추가로 계약 모호성 확인·재개, 프로세스 재시작, API 오류, 조직 간 접근, 오래된 승인, 저장 후 응답 유실을 검증한다. Harness 8요소별 테스트와 trace를 증거로 남긴다.

현재 상태와 완료 기준

현재 저장소에는 파싱·비교·분석·기본 그래프와 Gradio가 있다. MCP, 인증, 영속 승인·재개, 버전 저장, 작업 큐는 추가 개발 대상이다. README 기준 실제 DART 원문 파싱은 미검증이다. 목표 완료는 설계한 분기·도구 선택·승인·복구가 실제 실행되고 평가표 항목별 증거가 제출된 상태다.

11 / 발표 및 제출 구성

평가자가 확인할 산출물

평가표를 설명 자료·구현 코드·실행 결과로 연결해 제출한다.

발표 순서

① 고객의 수정 누락 문제와 B2B 가치 → ② 입력·출력 예시 → ③ 전체 아키텍처 → ④ StateGraph의 조사·확인·승인 흐름 → ⑤ 두 MCP 서버의 실제 호출 → ⑥ Harness·인증·복구 검증 → ⑦ 정량 평가와 남은 한계.

제출 파일 제안

아키텍처·상태 전이 문서, MCP 도구 명세, 인증·권한 표, Harness 8요소 구현 매핑, 테스트 결과, 평가 데이터 명세·보고서, 대표 trace, 실행 README와 환경변수 예시를 제출한다. 키 없는 합성 데모와 실제 연동 실행 방법을 구분한다.

발표용 설명

“저희는 정정공시가 기존 분석 메모의 어떤 주장에 영향을 주는지 찾아주는 서비스를 구현합니다. Agent는 필요한 공시와 근거를 MCP로 조회하며, Harness가 도구 권한과 실행 한도, 복구를 관리합니다. 수정안은 사람의 승인을 받은 뒤 버전을 검증해 저장하므로, 분석뿐 아니라 검토와 변경 이력까지 이어지는 업무 흐름을 제공합니다.”

문서 근거와 설계 상태

요구사항: 사용자가 제공한 ‘팀별 Capstone 평가표’와 ‘Harness Summary’ 이미지.
프로젝트 근거: PROJECT_BRIEF.md, CODEX_PROMPTS.md, disclosure-impact-agent/README.md, src/disclosure_impact_agent/graph.py.
본 문서는 평가 기준을 반영한 구현 설계 초안이다. 외부 API·라이브러리의 최신 사양 검증과 기능 구현은 후속 개발에서 수행하며, 이 문서 작성 과정에서 기능 테스트를 새로 실행하지 않았다.